

UNIT-IV

Feature-Driven Development:

Introduction

Incremental Software Development

Regaining Control:

The Motivation behind FDD

Planning an Iterative Project

Architecture Centric,

FDD and XP

Feature-Driven Development

- One solution to control the complexity inherent in agile, incremental projects is to apply a feature-centric process. A feature-centric process is one that tries to provide a way for management to handle questions such as:
 - What must we do next to add value to the client?
 - How are we progressing against time and budgets?
 - What issues and risks does the project face?
 - How can the issues and risks be addressed or mitigated?
 - What should we do next?

Feature-centric processes do this while retaining the motivations behind the agile movement such as:

-
- Individuals and interactions over processes and tools.
- Working software over comprehensive documentation.
- Customer collaboration over contract negotiation.
- Responding to change over following a plan.

With the key aims being:

- To satisfy the customer.
- Deliver working software that adds value to the customer.
- Working software is the primary measure of progress.
- Promote sustainable development.
- Keep any process as simple as is reasonably possible.

The term feature-centric refers to development processes that attempt to focus on combining the units of requirements, with the units of planning and the units of work.

This allows:

- the things users want,
- to be planned for and monitored,
- and to be used as the basis of work allocation.

Incremental Software Development

When applying some form of iterative approach, the intention is that each iteration adds something to the evolving system. Some iterations may lead to a release of the software system, while others may not. Each iteration:

- Determines what will be done during the iteration.
- Designs and implements the required functionality.
- Tests the new functionality.
- (Optionally) Creates a new release.
- Reviews what has been done before moving to the next iteration.

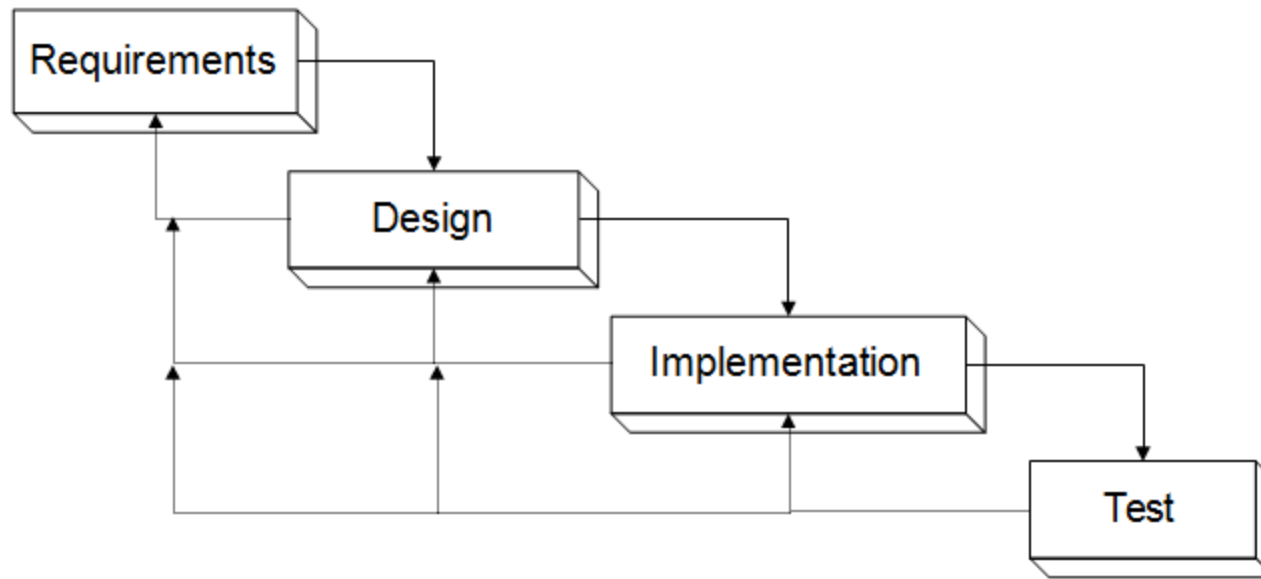


Fig. 9.1 The waterfall model.

One of the features of the waterfall model of software engineering used by many design methods (see Figure 9.1) is that it primarily assumes that you will complete the requirements analysis before you start the design phase. In turn, you will complete the design phase before you start the implementation phase, and so on

Let us review what happens during an incremental software development project. An incremental software development process is one that does not try to complete the whole design task in one go. This is in contrast to the more traditional waterfall model of software development.

Figure 9.2 depicts the spiral nature of this approach to software development. Note that in effect, each iteration around the spiral is a mini-software development project.

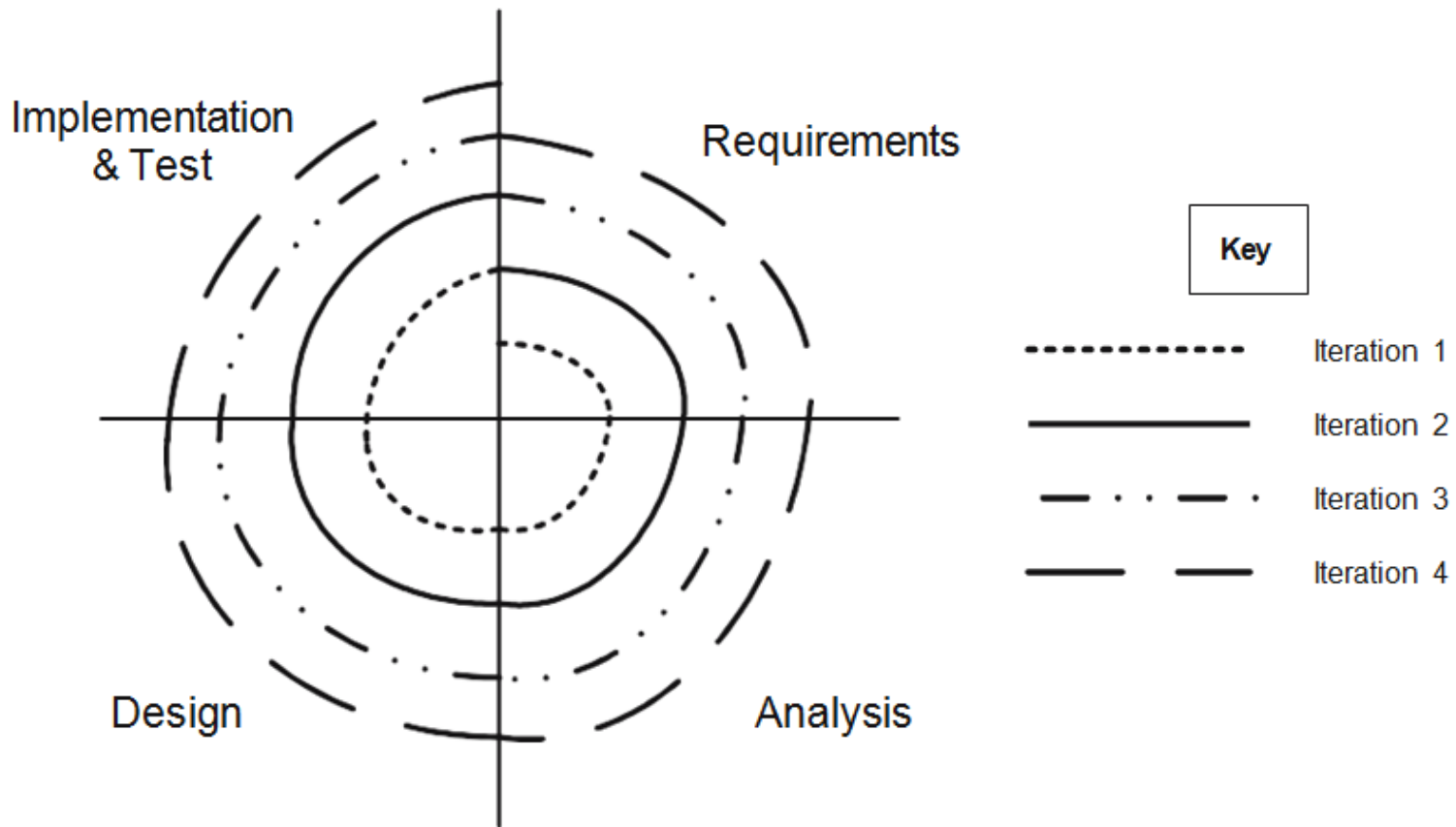


Fig. 9.2 Spiral model of software development.

Regaining Control: The Motivation Behind FDD

Given that our goal is to simplify the lifecycle in order that we can deal with the risks and complexities as well as uncertainties of the development process, we need to regain some control of the planning and management aspects of such a project. A key strategy for this is feature-centric planning. Feature-centric is not the only aspect of regaining control of an iterative project; another feature is that of timeboxing each iteration. The final aspect is being adaptive.

Thus, to regain control of an iterative project the guidelines are:

- The process should be *feature-centric*. This means that the units of requirements (e.g., use cases, user stories) should be unified with the units of planning (e.g., work packages and tasks).
- Project planning should be based around *timeboxes* (rather than phases) so that the length of each iteration is known.
- The project plan should be *adaptive* that is responsive to the changing risks and benefits of the system and business environment.

Feature-Centric Development

The term feature-centric is used to refer to development processes that combine the expression of requirements with the units of activity for planning purposes. A feature in such a process can be viewed as a unit of “plannable functionality.”

A feature is a schedulable piece of functionality, something that delivers value to the user. Note the emphasis on schedulable. That is, a feature is derived from a planning perspective rather than from the user perspective or indeed the requirements perspective. This is an important distinction and why features differ from requirements, user stories or use cases (even if they are derived from them).

To aid in planning, features go further, they must also be associated with:

- a priority (so that they can be ordered),
- a cost (so that they can be accounted for),
- resources (so that they can be scheduled).

Costs and resources can be determined by examining the number of person days taken to accomplish the feature. Priority can be harder to determine but should take into account:

- Architectural importance of the feature.
- Utility to the user.
- Risk involved.
- Requirements of the system/use cases.

To illustrate what I mean by a feature, consider the requirement “Add a New File” to an application. This might be described by a user story that explains how the user might add a new file to a file auditing system. The user story describes

this activity from the perspective of the user (obviously) and without reference to resource or costs (although user stories do have a relative priority). From this we can extract one or more system features that will be needed to implement this requirement. For example, these might be:

- “New File Dialog” feature.
- Java business objects for new file creation feature.
- New file details XML writer.

Timeboxing Iterations

Rather than defining each iteration by the features it will implement, it is possible to define an iteration in terms of the time period it will take and the ordered list of features that will be attempted during the time period. Features that are lower down the priority list will only be attempted if time allows, otherwise they will be relegated to a later iteration.

Timeboxing iteration has a number of benefits including the ability to:

- schedule and plan for incremental releases of the software,
- schedule and implement features,
- manage budgets,
- monitor progress within fixed time constraints.

Being Adaptive but Managed

An iterative project is effectively planned and re-planned at each stage of the spiral presented back in Figure 9.2. There is an overall planning activity before the whole process starts and then there are planning activities at the start of each iteration. In addition, regular (weekly) reviews may also affect the current plan for an iteration.

Planning an Iterative Project

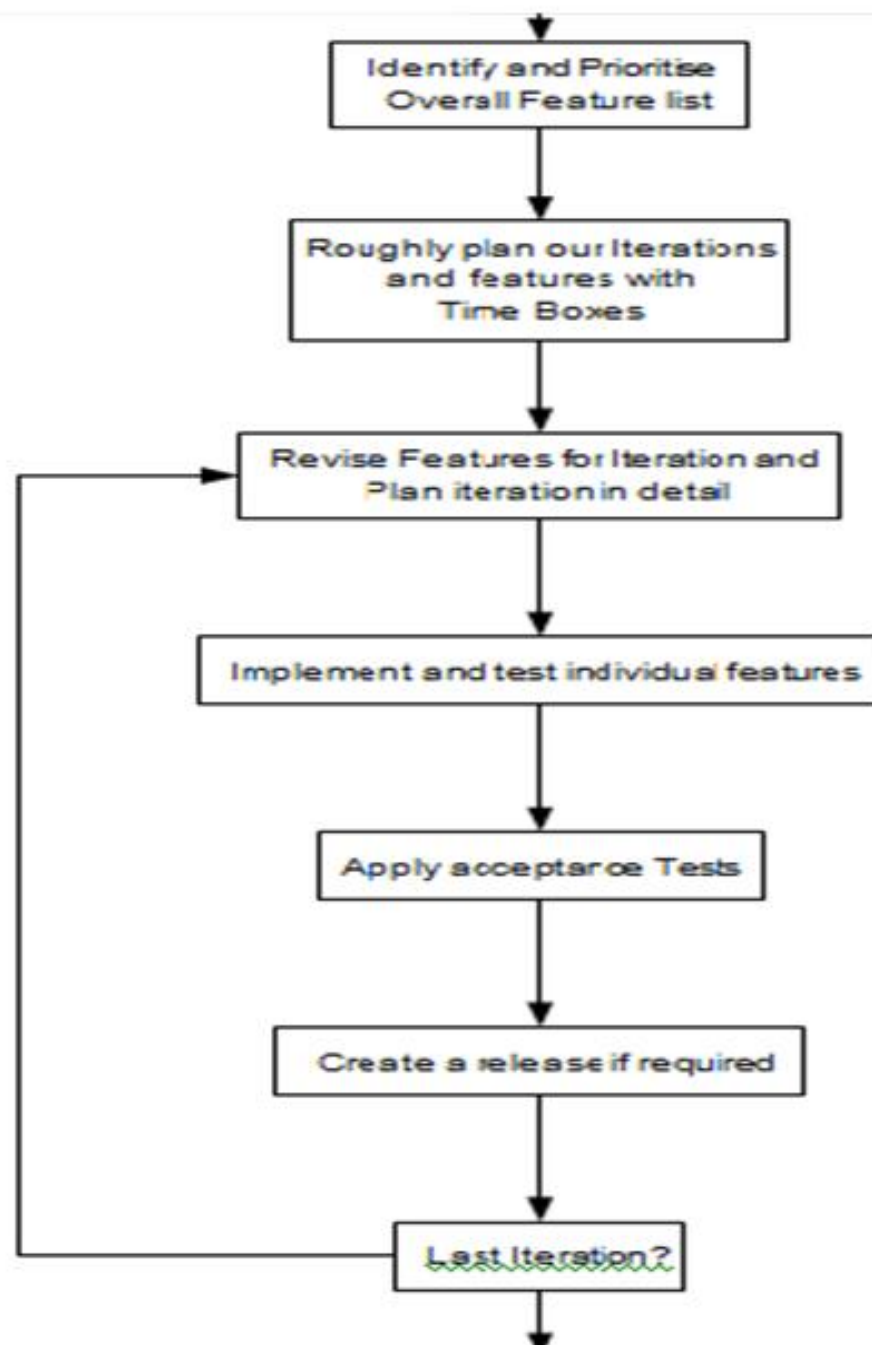
Before any project embarks on an iteration development process there are a number of steps that should be followed.

These steps are:

- Identify and priorities features (the feature list should be continually revised throughout the project).
- Roughly identify iterations and allocate features. Time box iterations/calculate costs. For each iteration
- Plan iterations (which should be continually revised during lifetime of project).
- Identify tasks required to implement features.
- Allocate tasks to resources (that is, allocate tasks to project members).
- Implement iteration.

Iterations, Timeboxes and Releases

- First let us review the overall project planning process. This is illustrated in Figure 9.3. We will discuss this figure in more detail below.
- At the start of the project, the project team along with various project stake- holders create a prioritized feature list. Note that this cannot be done without the collaboration of those project stakeholders who can state what the priorities of the features should be. Thus, as with XP, the “business” is an essential part of this process.



- As an example, consider the following iteration plan, taken from an actual project of a few years ago. There are five iterations in total for a legal advice expert system. The first iteration aimed to explore the problem domain and to produce a prototype expert system based on a combination of a simple Case-Based Reasoning system and a Rule-Based Reasoning system. The second iteration aimed to fill out this prototype with a larger amount of domain knowledge to confirm the viability of the approach. Iteration 3 then aimed to produce a commercial quality version of the initial prototype. Iteration 4 introduced a variety of management information system aspects to the system to review results, advice given, problems not dealt with, etc. The final iteration would address user feedback and any deployment issues encountered earlier in the project.
- The timing for the iterations were
- **Iteration 1:** Analyse and Prototype
- 58 person days. April 2003 – end of June 2003
- **Iteration 2:** Prototype II
- 119 person days. July 2003 – start of December 2003
- **Iteration 3:** Pilot and Develop
- 55 person days. December 2003 – end of February 2004
- **Iteration 4:** Develop II
- 41 person days. March 2004 – April 2004
- **Iteration 5:** Final Release
- 16 person days. End of April 2004 – mid May 2004.

ID	Task Name	Start	End	Duration	Q3 03			Q4 03			Q1 04			Q2 04					
					Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May	
1	Analyse and Prototype	07/04/2003	25/06/2003	58d															
2	Prototype II	26/06/2003	09/12/2003	115d															
3	Pilot and Develop	10/12/2003	24/02/2004	55d															
4	Develop II	25/02/2004	21/04/2004	41d															
5	Final Release	22/04/2004	13/05/2004	16d															

Planning an Iteration

- Each iteration will be comprised of a similar set of steps. These steps are presented graphically in Figure 9.4.
- The key steps in any iteration are:
- *Iteration initiation meeting.* The length of the iteration should already have been determined but may be revised at this point. The features to be addressed in this iteration should be revised and confirmed along with the resources to be applied, etc. This meeting should involve all stakeholders in the project.

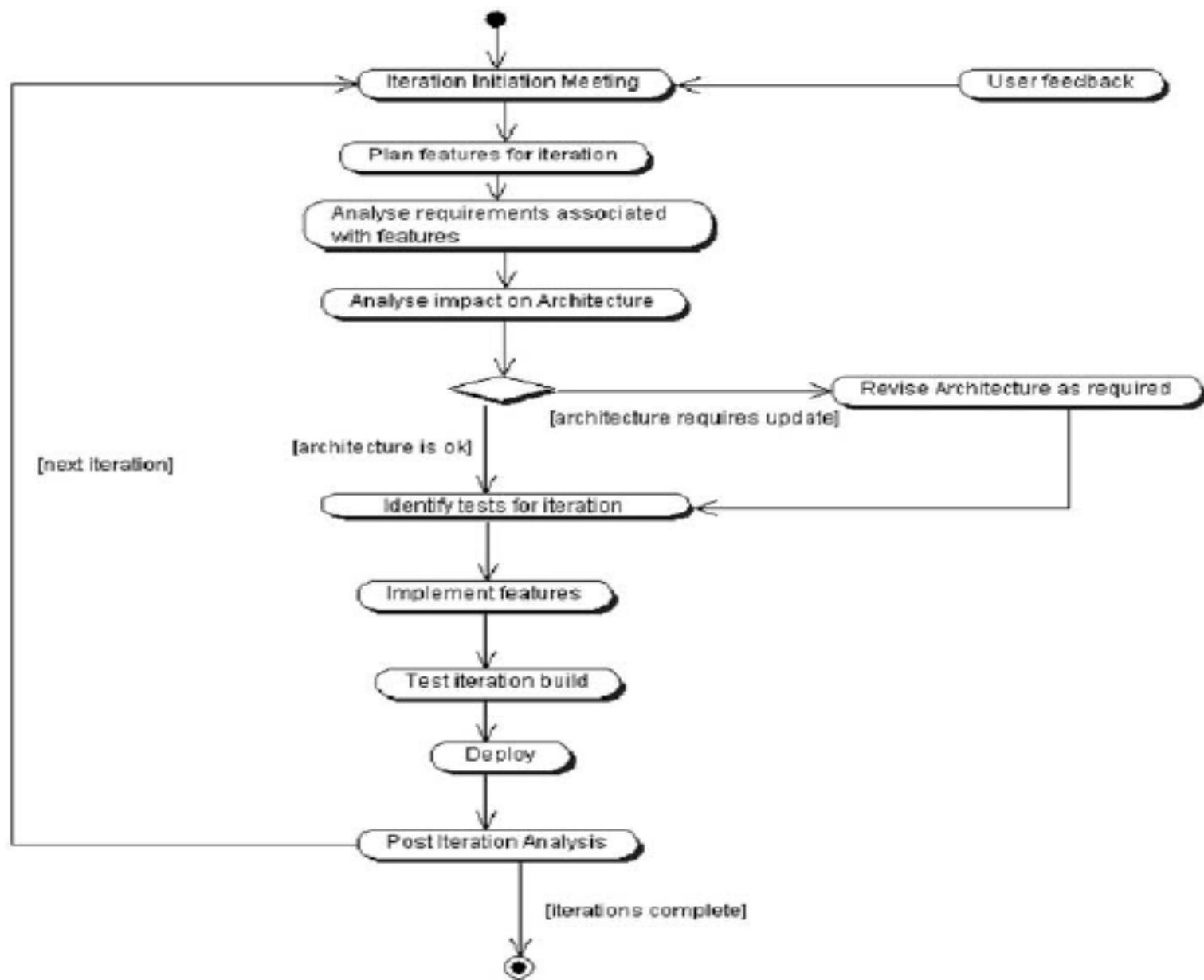


Fig. 9.4 The steps within an iteration.

- *Plan features for iteration.* Having agreed the features to be addressed, a detailed plan should be produced mapping features to work packages and work packages to tasks. The tasks in turn should be allocated to actual re- sources, etc.
- *Analyse the requirements associated with the features.* This may involve writing or revising a use case document, designing new GUI displays, determining the user interaction sequence, etc. The acceptance criteria for this iteration should also be identified and agreed.
- *Analyse impact on the architecture.* The architecture is the backbone upon which the iterative process operates, therefore the next step to perform is to examine the impact any new features are likely to have on the architecture. It may also involve identifying new architecturally significant entities that feed into the next step.
- *(Optionally) Revise architecture as required.* This step involves revisiting and amending the application architecture in response to the features required by this implementation. Note that this may mean that some of the design and analysis work associated with core features may be performed at this stage to determine their architectural impact.
- Next a *new acceptance test plan and specification* should be written for this iteration. Note this document may not include all tests as some features may only be implemented if time allows. The specification of the tests for these features should be considered to be a task within the work package that will address that feature.

- The next step involves *implementing the features*. The features are actually implemented via tasks that should be monitored as normal (although reference should be made to the timebox of the iteration). It is recommended that each feature should have a set of associated unit tests that must be passed before the feature is taken to have been completed. These unit tests should be part of the code released for the feature and ideally should be added to a unit test framework (such as JUnit) that can easily be re-run at regular intervals (such as every time a new build of the system occurs).
- Once the features are implemented, the new *system should be tested* (this includes the generation of a test report). This includes unit tests and acceptance tests. All tests should pass before the iteration is allowed to proceed. If any tests fail, then the release cannot be deployed and the problems must be corrected.

- The new application should then be *deployed* to the client who should then perform any agreed user acceptance tests. This may lead to the revision of the deployed system, if and when deficiencies are identified.
- A *post iteration meeting* should review the progress made during the iteration, it should consider any issues that arose and re-prioritise any features that were not addressed. Again, this should involve all project stakeholders.
- At this point, a decision should also be made regarding the *validity of the next* iteration and whether any further iterations are required.

An Aside on Planning within an FDD Project

I have probably been involved in more work managed using feature-driven development (FDD) than any of the agile methods. And one comment made by one of the clients sticks out in my mind. We had worked on a system over a couple of years, with at least six iterations developed and successfully released. Near the end of this project, she commented to me one day, that the thing that had surprised her the most was the amount of planning we had done on the project

Estimating the Cost of a Feature

The approach I have taken over the last few years towards estimating the cost of features is heavily influenced by the approach described by Carmichael and Haywood (2002). This approach involves applying three-point estimating. Three- point estimating involves producing three estimates of the effort that will be required to implement a feature. These are:

- *The best-case scenario.* This represents the situations where everything goes as well as it could and there are absolutely no surprises or problems encountered.
- *The best guess at what it will really take.* This represents the situation where mostly everything goes okay, but one or two unexpected situations occur which take a little longer than originally expected to handle.
- *The worst-case scenario.* This represents the situation where major issues were overlooked (because they were not obvious until implementation started).

	A	B	C	D	E	F
1						
2						
3	Feature	Priority	Best Case	Likely Case	Worst Case	Estimate (person days)
4	Assessment summary page	H	1	2	4	2.17
5	Management of Not Auditable Files	H	3	4	7	4.33
6	Remove account manager field	H	1	1	2	1.17
7	Copy / cut / paste on screens	H	1	2	5	2.33
8	Order of summary trends	H	3	5	10	5.50
9	Formatting of text	L	3	5	11	5.67
10	Lock cross reference key	H	1	3	4	2.83
11	Support for London Weighting	M	2	4	5	3.83
12	Summary report modifications	L	2	3.5	6	3.67
13						
14		Person days	17	29.5	54	31.50

Fig.9.5 Three-point estimating for a list of features.

- A table illustrating this style of estimating for a list of features is illustrated in Figure 9.5. The table illustrates the feature, its priority and the three estimates for best, most likely and worst-case scenarios.
- The overall estimate for each feature is derived by applying a formula in the underlying spreadsheet. There are various formulas that could be applied; in this case we have applied the following formula:
 - $(\text{Best case} + (4 \times \text{likely case}) + \text{worst case})/6$
- This formula gives weight to the most likely case but also takes into account the range from the best and worst cases. It is possible to take this further and use a pseudo standard deviation to gain an estimate of how reliable the numbers are.

Architecture Centric

Feature-driven development or feature-centric development (FCD) is only feasible (and successful) if there is a solid architecture on which each iteration can be built. This section examines what is meant by an architecture and why it is central to a successful iterative process.

Why Architecture Centric?

One problem with an iterative and incremental approach is that if no order or structure was defined for the application it could (would?) grow more and more unwieldy and more and more disorganised as each iteration progressed. To ensure that all the various parts fit together, there needs to be something. That something is the architecture. The architecture is the skeleton on which the muscles (functionality) and skin (the user interface) of the system will be hung. A good architecture will be resilient to change and to the evolving design and implementation

Architecture Defined

A software architecture encompasses

- *the overall plan for the structure of the system.* That is, it is the blueprint for the system. It should be possible to read the architecture and gain an appreciation for how the system was structured (without needing to know the details of the structural elements)
- *the key structural elements and their interfaces.* That is what with which elements make up the system, their interfaces and how they are plugged together. *how those elements interact (at the top level).* That is, when the various elements of the architecture interface, what do they do and why do they

- *how these elements form subsystems and systems.* This is a very important aspect of the architecture. Early identification of the core systems and subsystem of the design not only helps organize future design (and implementation) work, it helps promote reuse and the comprehensibility of the system.
- *architectural style* that guides this project.

Why Have an Architecture?

- Let us review the argument being made about why an architecture is a critical element of the object-oriented design process. We need an architecture to
- *understand the system.* Software systems can be large, complex and must meet conflicting requirements. An architecture provides a convenient blueprint or model of the system to be produced. It abstracts out much of the implementation detail, but “positions” the elements that must meet the various functional requirements.
- *organise development.* It helps organise “plumbers” and “electricians.” That is, it helps firstly to separate out different concerns so that those involved in the “plumbing” of the system only need to worry about plumbing issues. However, it also identifies how they are related, so that the points at which different concerns intersect, are well documented and clearly specified (for example, in the central heating boiler). *promote reuse.* The problem with writing reusable code is that you need to identify that what you are producing is reusable. I have personally been in situations where two people on one project are reproducing the same solution but from different aspects.

FDD and XP

Features can be implemented in a variety of different ways using a variety of techniques. However, taking an agile approach means that applying the techniques we have discussed so far in this book can work extremely well. Within a development model in which FDD is used to plan the details of iterations and in which features are treated as the tasks to be performed, then applying Agile Modeling and XP practices can result in a workflow resembling that presented in Figure 9.7.

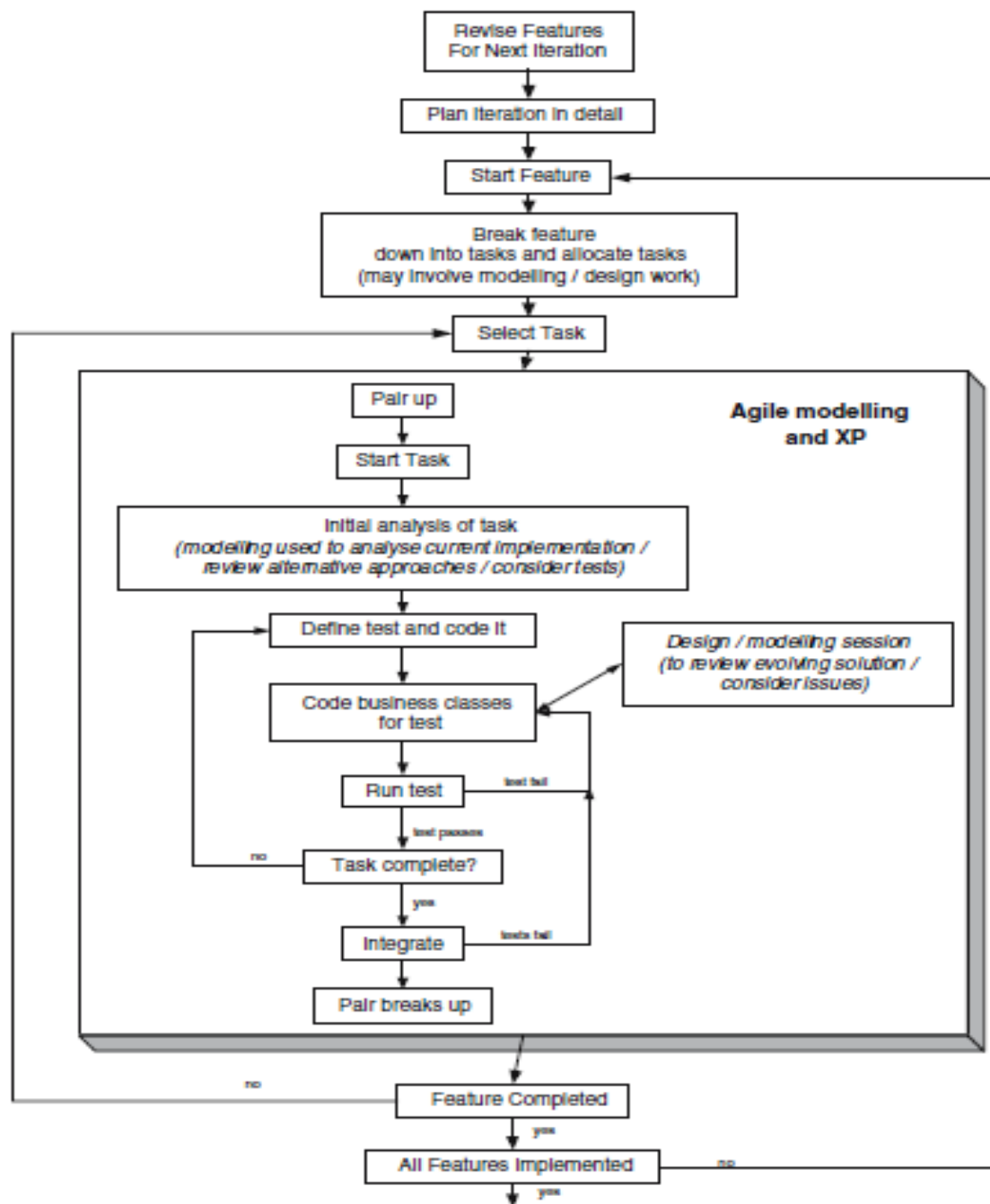


Fig. 9.7 Combining FDD with Agile Modelling and XP.

UNIT-IV

- 1.What are the guidelines regain control of an iterative project.
2. What are Time boxing iteration ? & the number of benefits ?
- 3.Explain the steps in Planning an Iterative Project in detail with an neat diagram.
- 4.Explain the no of steps in an each iteration with an example.
- 5.What are the *case scenario* for Estimating the Cost of a Feature.
- 6.Define Architure & why we need an Architure.?
- 7.Explain the Combining FDD with Agile Modeling and XP.
- 8.What is Feature-Driven Development (FDD)?
- 9.What are the main goals of Feature-Driven Development?
- 10.What are the key characteristics that define FDD?
- 11.What are the key differences between the practices of FDD and XP?
12. Why is a well-defined architecture crucial for the success of an FDD project?